

Lab 1: [Python Exercise]

McKenna Taylor (747408407)

2026-08-05

Table of contents

Setup	1
Part A: [Tracer Decay and Set Up Size (numerical)]	1
Part B: [Resivor Box Model (process-based)]	8
Reflection	15

Setup

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

Part A: [Tracer Decay and Set Up Size (numerical)]

This shows the forward-Euler method for solving the ordinary differential equation for tracer decay. The exact solution is also shown for comparison.

```
def decay_numerical(C0, k, dt, t_end):
    """Return arrays of time and concentration for forward-Euler tracer decay."""
    n = int(t_end / dt)
    C = C0
    ts = [0.0]
    Cs = [C0]
    for i in range(n):
        C = C + (-k * C) * dt          # forward difference
        ts.append((i + 1) * dt)
```

```

        Cs.append(C)
    return np.array(ts), np.array(Cs)

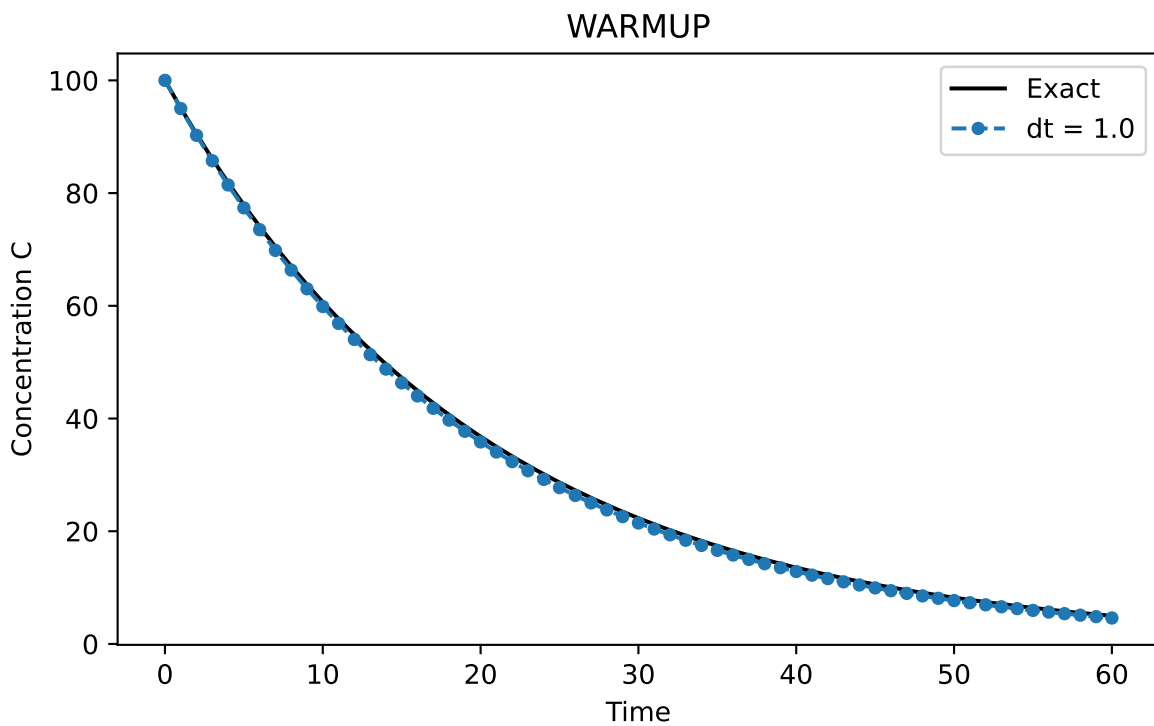
# The exact answer, for comparison
C0, k, t_end = 100.0, 0.05, 60
t_fine = np.linspace(0, t_end, 200)
exact = C0 * np.exp(-k * t_fine)

fig, ax = plt.subplots(figsize=(7, 4))
ax.plot(t_fine, exact, "k-", label="Exact")

for dt in [1.0]:
    ts, Cs = decay_numerical(C0, k, dt, t_end)
    ax.plot(ts, Cs, "o--", markersize=4, label=f"dt = {dt}")

ax.set_xlabel("Time")
ax.set_ylabel("Concentration C")
ax.set_title("WARMUP")
ax.legend()
plt.show()

```



```

def decay_numerical(C0, k, dt, t_end):
    """Return arrays of time and concentration for forward-Euler tracer decay."""
    n = int(t_end / dt)
    C = C0
    ts = [0.0]
    Cs = [C0]
    for i in range(n):
        C = C + (-k * C) * dt          # forward difference
        ts.append((i + 1) * dt)
        Cs.append(C)
    return np.array(ts), np.array(Cs)

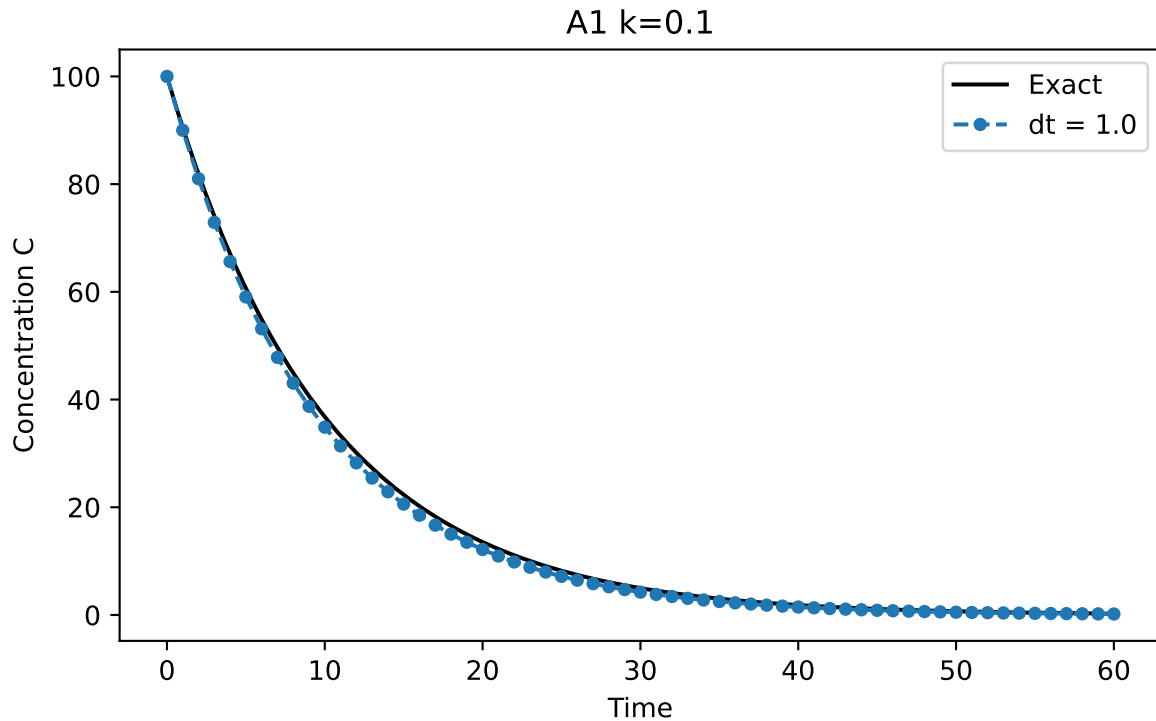
# The exact answer, for comparison
C0, k, t_end = 100.0, 0.1, 60
t_fine = np.linspace(0, t_end, 200)
exact = C0 * np.exp(-k * t_fine)

fig, ax = plt.subplots(figsize=(7, 4))
ax.plot(t_fine, exact, "k-", label="Exact")

for dt in [1.0]:
    ts, Cs = decay_numerical(C0, k, dt, t_end)
    ax.plot(ts, Cs, "o--", markersize=4, label=f"dt = {dt}")

ax.set_xlabel("Time")
ax.set_ylabel("Concentration C")
ax.set_title("A1 k=0.1")
ax.legend()
plt.show()

```



```
def decay_numerical(C0, k, dt, t_end):
    """Return arrays of time and concentration for forward-Euler tracer decay."""
    n = int(t_end / dt)
    C = C0
    ts = [0.0]
    Cs = [C0]
    for i in range(n):
        C = C + (-k * C) * dt          # forward difference
        ts.append((i + 1) * dt)
        Cs.append(C)
    return np.array(ts), np.array(Cs)

# The exact answer, for comparison
C0, k, t_end = 100.0, 0.01, 60
t_fine = np.linspace(0, t_end, 200)
exact = C0 * np.exp(-k * t_fine)

fig, ax = plt.subplots(figsize=(7, 4))
ax.plot(t_fine, exact, "k-", label="Exact")

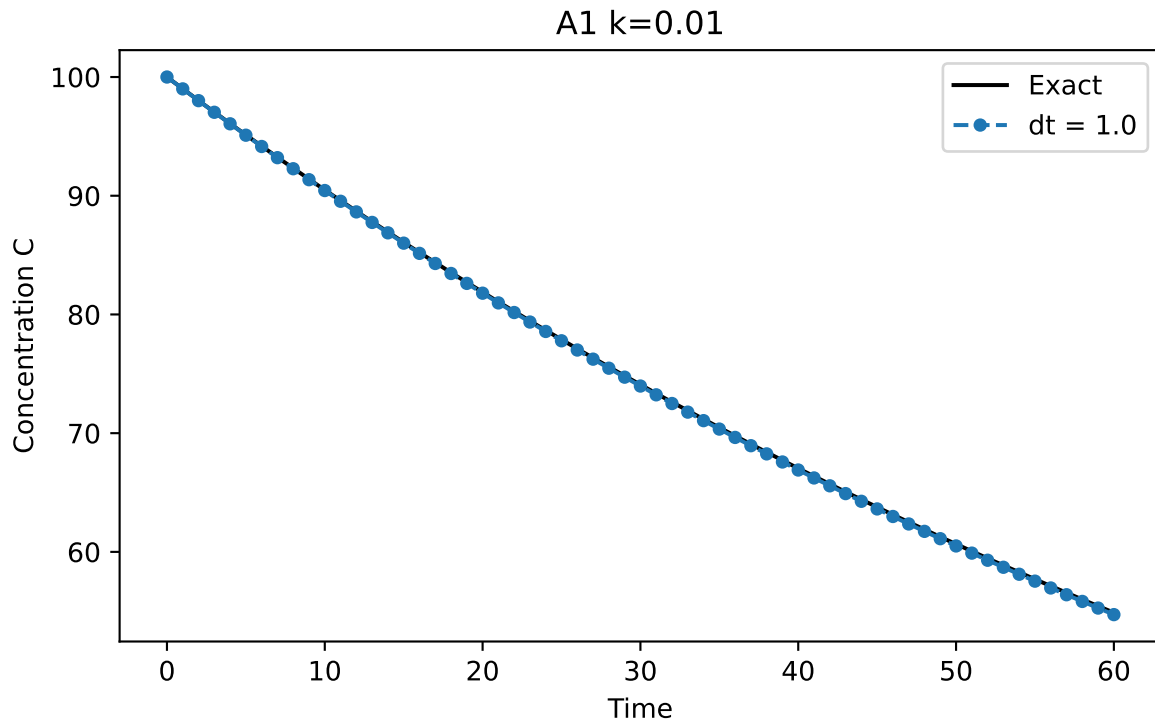
for dt in [1.0]:
```

```

ts, Cs = decay_numerical(C0, k, dt, t_end)
ax.plot(ts, Cs, "o--", markersize=4, label=f"dt = {dt}")

ax.set_xlabel("Time")
ax.set_ylabel("Concentration C")
ax.set_title("A1 k=0.01")
ax.legend()
plt.show()

```



```

def decay_numerical(C0, k, dt, t_end):
    """Return arrays of time and concentration for forward-Euler tracer decay."""
    n = int(t_end / dt)
    C = C0
    ts = [0.0]
    Cs = [C0]
    for i in range(n):
        C = C + (-k * C) * dt          # forward difference
        ts.append((i + 1) * dt)
        Cs.append(C)
    return np.array(ts), np.array(Cs)

```

```

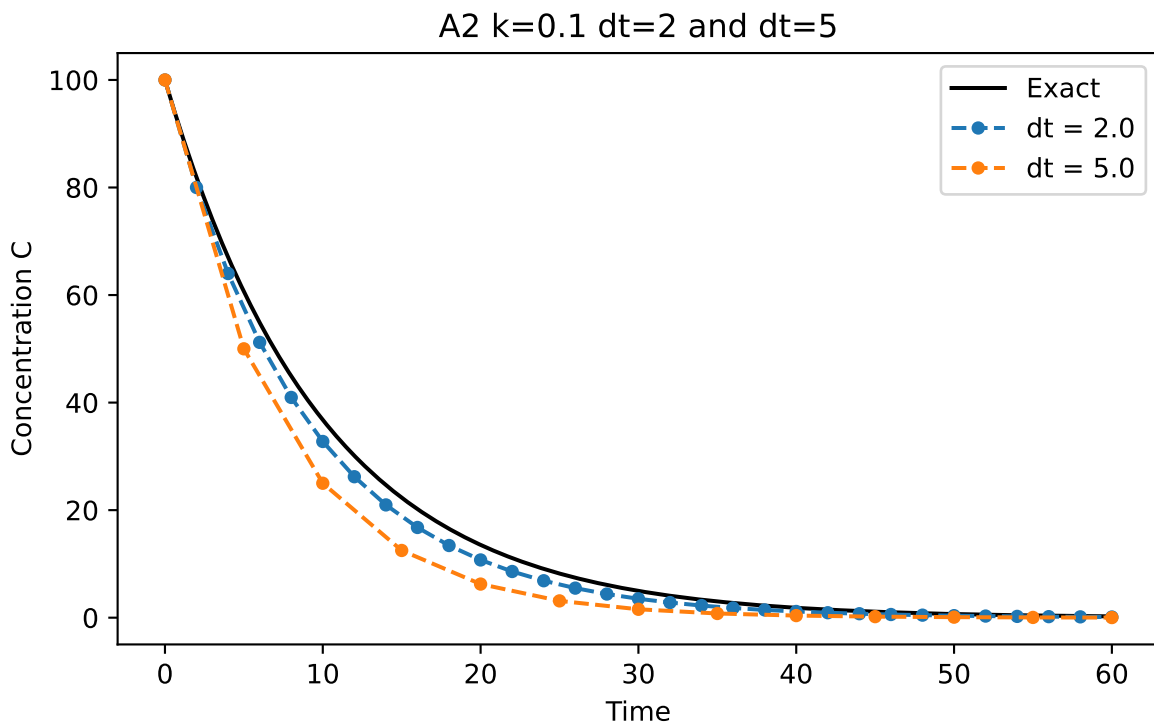
# The exact answer, for comparison
C0, k, t_end = 100.0, 0.1, 60
t_fine = np.linspace(0, t_end, 200)
exact = C0 * np.exp(-k * t_fine)

fig, ax = plt.subplots(figsize=(7, 4))
ax.plot(t_fine, exact, "k-", label="Exact")

for dt in [2.0, 5.0]:
    ts, Cs = decay_numerical(C0, k, dt, t_end)
    ax.plot(ts, Cs, "o--", markersize=4, label=f"dt = {dt}")

ax.set_xlabel("Time")
ax.set_ylabel("Concentration C")
ax.set_title("A2 k=0.1 dt=2 and dt=5")
ax.legend()
plt.show()

```



```

def decay_numerical(C0, k, dt, t_end):
    """Return arrays of time and concentration for forward-Euler tracer decay."""
    n = int(t_end / dt)
    C = C0
    ts = [0.0]
    Cs = [C0]
    for i in range(n):
        C = C + (-k * C) * dt          # forward difference
        ts.append((i + 1) * dt)
        Cs.append(C)
    return np.array(ts), np.array(Cs)

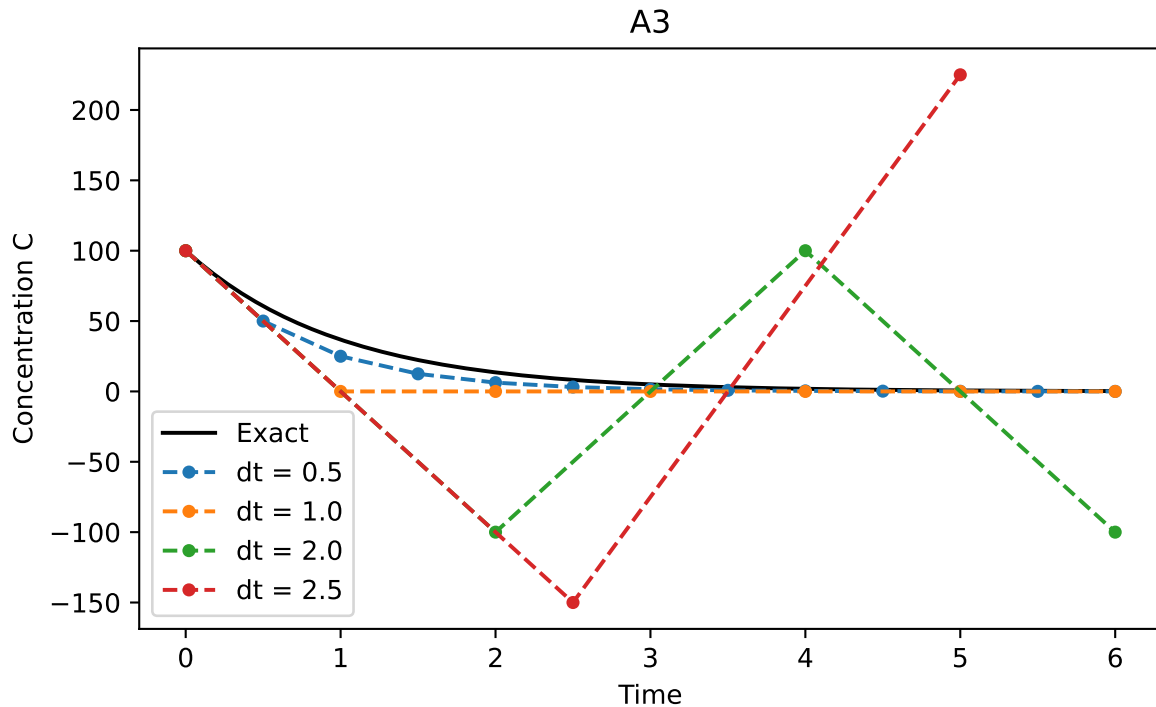
# The exact answer, for comparison
C0, k, t_end = 100.0, 1.0, 6.0
t_fine = np.linspace(0, t_end, 200)
exact = C0 * np.exp(-k * t_fine)

fig, ax = plt.subplots(figsize=(7, 4))
ax.plot(t_fine, exact, "k-", label="Exact")

for dt in [0.5, 1.0, 2.0, 2.5]:
    ts, Cs = decay_numerical(C0, k, dt, t_end)
    ax.plot(ts, Cs, "o--", markersize=4, label=f"dt = {dt}")

ax.set_xlabel("Time")
ax.set_ylabel("Concentration C")
ax.set_title("A3")
ax.legend()
plt.show()

```



A1: k controls the concentrations decay rate over time; higher k value results in a faster decay, while a lower k value leads to a slower decay. Stays constant. A2: When running $dt=2$ and $dt=5$ with $k=0.01$ the rates follow the curve closely, but when $k=0.1$ both $dt=2$ and $dt=5$ deviate from nearly the same point - around 5 time and 65 concentration. However $dt=5$ decays more rapidly before evening out and meeting back with $dt=2$ for the rest of time. $dt=2$ decays at a slower rate and meets the exact curve at around 35 time and 5 concentration. A3: $dt=1$ is the largest without going negative. A4: If kdt is greater than 1 then we are trying to subtract more than 100% of the old concentrations to make a new concentration. This then puts us below zero and the new model becomes negative. The model is only valid for $kdt < 1$.

Part B: [Reservoir Box Model (process-based)]

Running models to understand how a lake responds to pollution inputs and decay processes.

```
import numpy as np
import matplotlib.pyplot as plt

# ---- Parameters ----
V = 1.0e6          # lake volume (m3)
Q = 5000           # river inflow (m3 per day)
C_in = 10.0        # incoming concentration (mg/L)
```



```

k = 0.05          # in-lake decay rate (per day)

# ---- Time settings ----
dt = 1.0          # time step (days)
days = 200

# ---- Initial condition ----
C = 20.0          # clean lake

# ---- Storage ----
history = [C]

# ---- Time loop ----
for _ in range(days):
    dCdt = (Q * C_in - Q * C - k * V * C) / V  # the rate of change
    C = C + dCdt * dt                          # forward difference
    history.append(C)

# ---- A quick look at the numbers ----
print(f"Started at {history[0]:.2f} mg/L")
print(f"After 10 days: {history[10]:.2f} mg/L")
print(f"After 50 days: {history[50]:.2f} mg/L")
print(f"After {days} days: {history[-1]:.2f} mg/L")

fig, ax = plt.subplots(figsize=(7, 4))
ax.plot(history)
ax.set_xlabel("Day")
ax.set_ylabel("Lake concentration (mg/L)")
ax.set_title("Reservoir box model: pollutant in a lake (B2)")
plt.show()

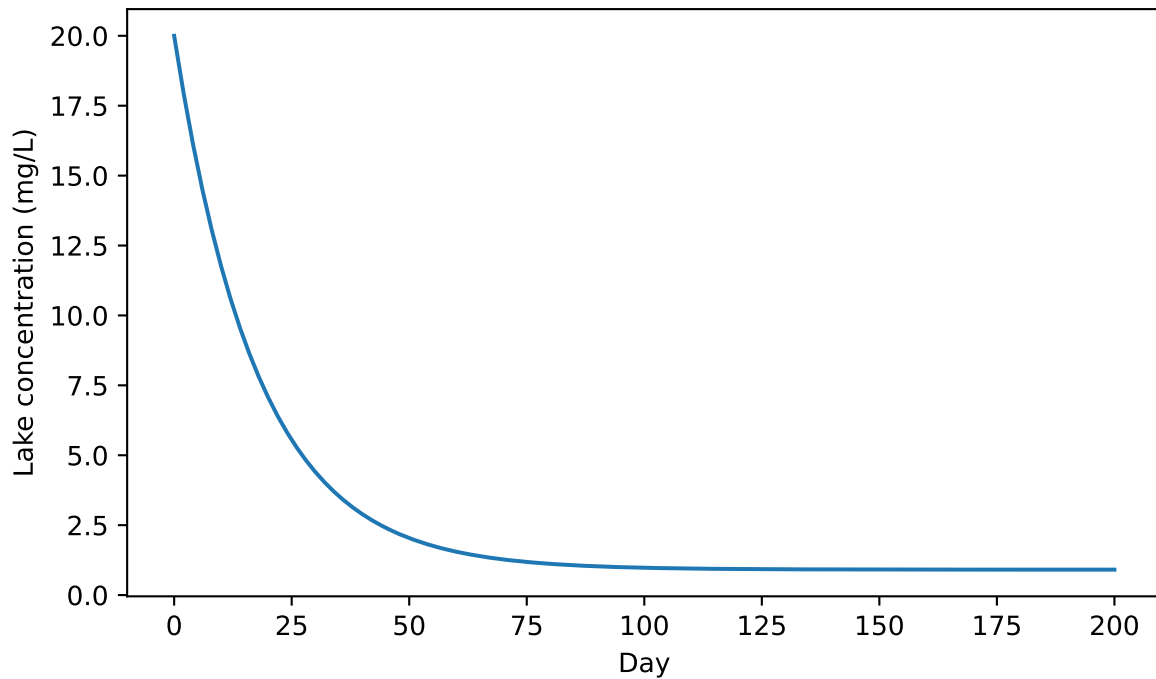
```

```

Started at 20.00 mg/L
After 10 days: 11.75 mg/L
After 50 days: 2.04 mg/L
After 200 days: 0.91 mg/L

```

Reservoir box model: pollutant in a lake (B2)



```
import numpy as np
import matplotlib.pyplot as plt

# ---- Parameters ----
V = 1.0e6      # lake volume (m^3)
Q = 10000     # river inflow (m^3 per day)
C_in = 10.0    # incoming concentration (mg/L)
k = 0.05      # in-lake decay rate (per day)

# ---- Time settings ----
dt = 1.0      # time step (days)
days = 200

# ---- Initial condition ----
C = 0.0      # clean lake

# ---- Storage ----
history = [C]

# ---- Time loop ----
for _ in range(days):
```

```

    dCdt = (Q * C_in - Q * C - k * V * C) / V    # the rate of change
    C = C + dCdt * dt                             # forward difference
    history.append(C)

# ---- A quick look at the numbers ----
print(f"Started at {history[0]:.2f} mg/L")
print(f"After 10 days: {history[10]:.2f} mg/L")
print(f"After 50 days: {history[50]:.2f} mg/L")
print(f"After {days} days: {history[-1]:.2f} mg/L")

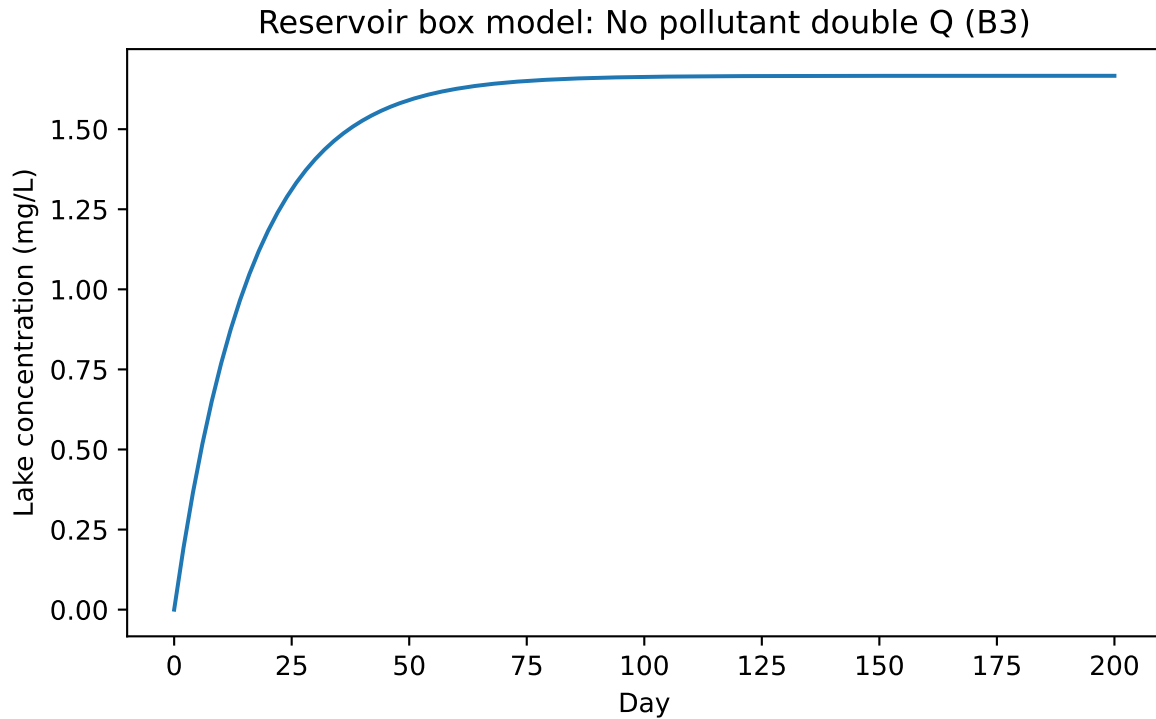
fig, ax = plt.subplots(figsize=(7, 4))
ax.plot(history)
ax.set_xlabel("Day")
ax.set_ylabel("Lake concentration (mg/L)")
ax.set_title("Reservoir box model: No pollutant double Q (B3)")
plt.show()

```

```

Started at 0.00 mg/L
After 10 days: 0.77 mg/L
After 50 days: 1.59 mg/L
After 200 days: 1.67 mg/L

```



```
import numpy as np
import matplotlib.pyplot as plt

# ---- Parameters ----
V = 1.0e6      # lake volume (m3)
Q = 5000      # river inflow (m3 per day)
C_in = 10.0    # incoming concentration (mg/L)
k = 0.05      # in-lake decay rate (per day)

# ---- Time settings ----
dt = 1.0      # time step (days)
days = 200

# ---- Initial condition ----
C = 0.0      # clean lake

# ---- Storage ----
history = [C]

# ---- Time loop ----
for _ in range(days):
```

```

    dCdt = (Q * C_in - Q * C - k * V * C) / V    # the rate of change
    C = C + dCdt * dt                            # forward difference
    history.append(C)

# ---- A quick look at the numbers ----
print(f"Started at {history[0]:.2f} mg/L")
print(f"After 10 days: {history[10]:.2f} mg/L")
print(f"After 50 days: {history[50]:.2f} mg/L")
print(f"After {days} days: {history[-1]:.2f} mg/L")

fig, ax = plt.subplots(figsize=(7, 4))
ax.plot(history)
ax.set_xlabel("Day")
ax.set_ylabel("Lake concentration (mg/L)")
ax.set_title("Reservoir box model: k change (B4)")
plt.show()

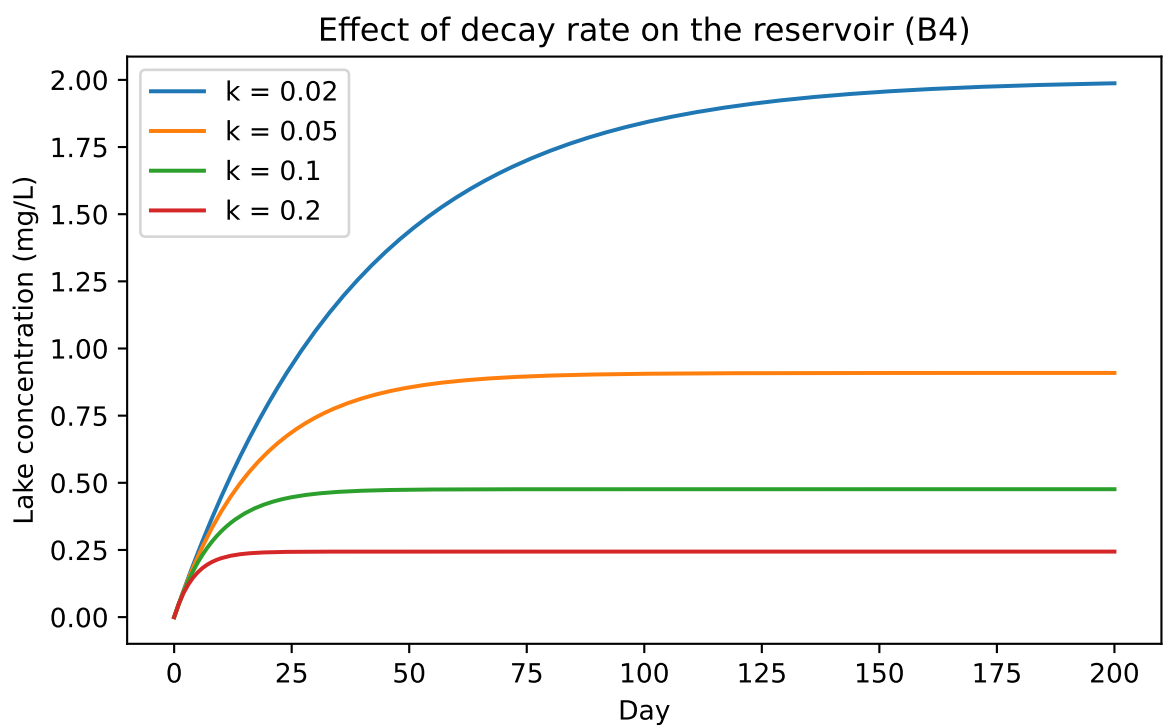
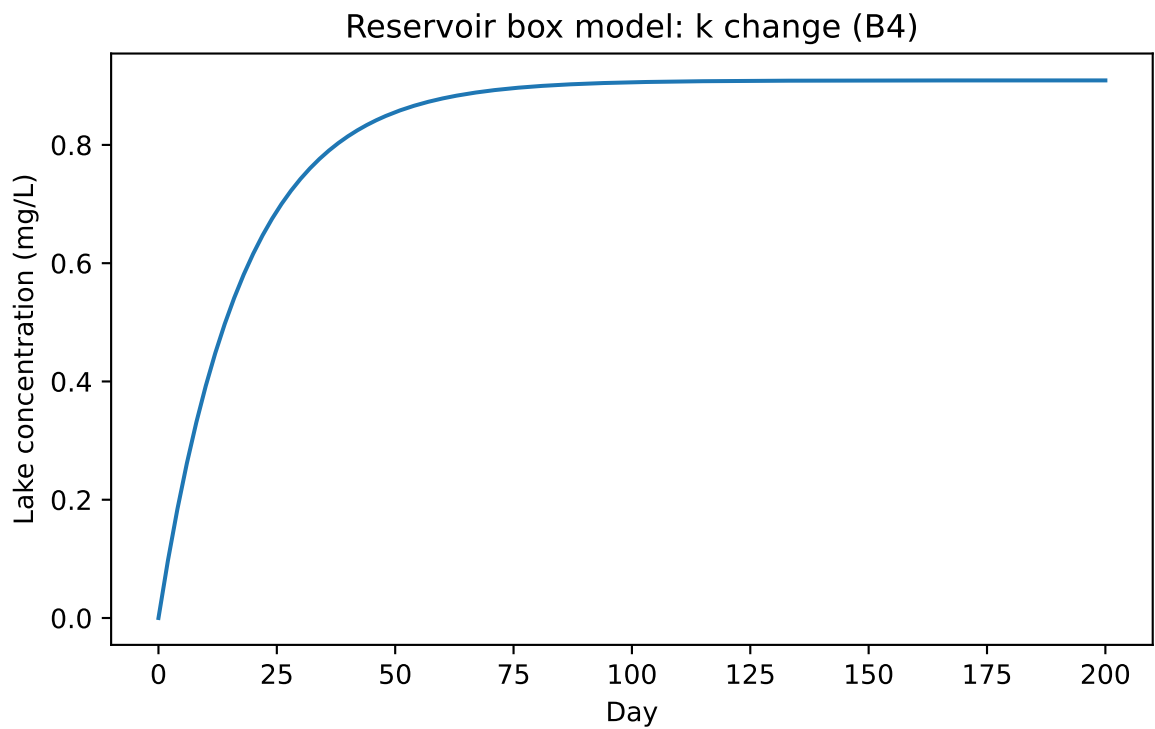
fig, ax = plt.subplots(figsize=(7, 4))

for k_test in [0.02, 0.05, 0.1, 0.2]:
    C = 0.0
    history = [C]
    for _ in range(days):
        dCdt = (Q * C_in - Q * C - k_test * V * C) / V
        C = C + dCdt * dt
        history.append(C)
    ax.plot(history, label=f"k = {k_test}")

ax.set_xlabel("Day")
ax.set_ylabel("Lake concentration (mg/L)")
ax.set_title("Effect of decay rate on the reservoir (B4)")
ax.legend()
plt.show()

```

Started at 0.00 mg/L
 After 10 days: 0.39 mg/L
 After 50 days: 0.86 mg/L
 After 200 days: 0.91 mg/L



B1: I am so confused ngl. Analytical steady state: 0.91 mg/L and numerical final value: 0.91 mg/L. B2: Over 200 days the concentration of the pollutant goes from 20 mg/L to 0.91 mg/L, which is a significant decrease, showing that the lake has the abilities to clean itself over time. Returning to its steady stay (0.91 mg/L) i thibnk. B3: The steady state concentraion raises from 0.91 mg/L to 1.67 mg/L. This makes sense as the inflow rate has doubled introducing more pollution to the lake. The final results of 1.67 mg/L is also constant. Where Q increases C increases because Q is fast which means C has less time to decay. B4: k effects the steady-state value by changing the rate of decay within the lakes and effects the speed of approach by decreasing the time to reach steady-state. higher k faster the decay and slower the steady-state approach.

Reflection

I got stuck on how to reopen this file after I had orignally closed it. I also got stuck on how to link this qmd to my github repo, I had to ask AI how to do that, but with the wise words of Hyesop guiding me I was able to do it. I also remembered almost at the end of the whole assignment that I needed to be making a new cell for each new question - I had just been changing one line of code and then running to model beucase I thought I would be able to submit the graphs... turns out nope this was a learning curve so I went back a redid all the code. I think beucase I had to go back an rerun all the code, I wouldn't change anything beucase I think I undertand how the code works now and I dont know enough about coding to even know where to start to change anything.